OOP Final Project

B123040023 戴維佑
B123040025 王勁詠
B123040026 張韶軒

Code Review

```
    learning_rate_a = 0.092 # alpha or learning rate
    discount_factor_g = 0.9 # gamma or discount rate. Near 0: more weight/reward placed on
mmediate state. Near 1: more on future state.
    epsilon = 1         # 1 = 100% random actions
    epsilon_decay_rate = 0.99965        # epsilon decay rate. 1/0.0001 = 10,000
    rng = np.random.default_rng()   # random number generator

    rewards_per_episode = np.zeros(episodes)

    for i in range(episodes):
        state = env.reset()[0]  # states: 0 to 63, 0=top left corner,63=bottom right corner
        terminated = False      # True when fall in hole or reached goal
        truncated = False       # True when actions > 200

        while(not terminated and not truncated):
            if is_training and rng.random() < epsilon:
                action = env.action_space.sample() # actions: 0=left,1=down,2=right,3=up
            else:
                action = np.argmax(q[state,:])

            new_state,reward,terminated,truncated,_ = env.step(action)
            if is_training and reward == 0 and terminated:
                reward = -1
```

Code Review

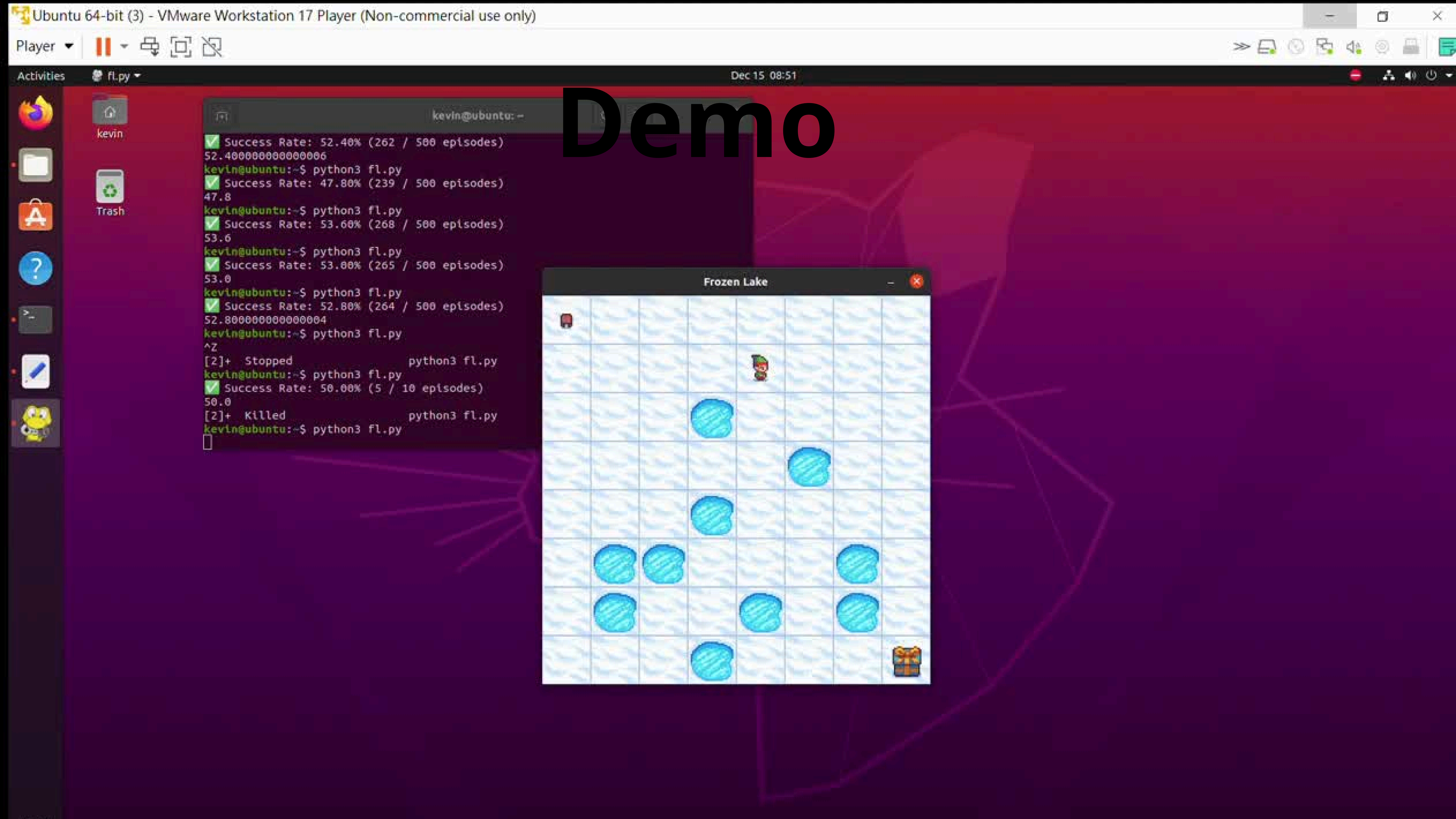
```python
if result["type"] == InteractionType.BLOCKED:
    reward = -2.0 # 撞牆懲罰加重

elif result["type"] == InteractionType.WIN:
    self.robot_pos = next_pos
    reward = 20.0
    terminated = True

elif result["type"] == InteractionType.SLIDE:
    self.robot_pos = next_pos
    dr, dc = result["dir"]
    slide_pos = [self.robot_pos[0] + dr, self.robot_pos[1] + dc]
```

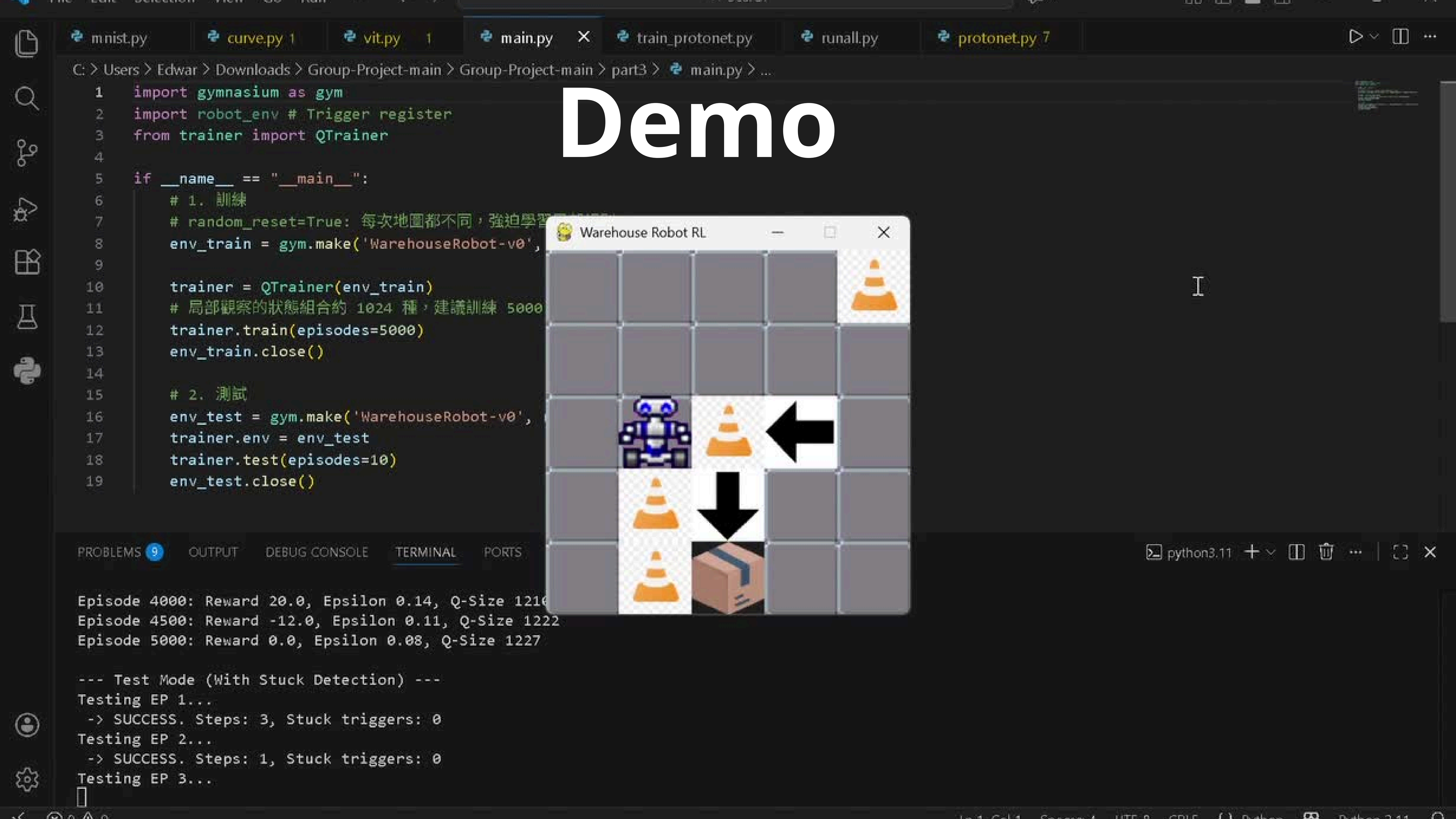
```python
def regenerate_map(self):
    """生成一張隨機但保證可解的地圖"""
    while True:
        self._generate_random_layout()
        if self._is_reachable((0, 0), self.target_pos):
            break
```

Demo



Ubuntu 64-bit (3) - VMware Workstation 17 Player (Non-commercial use only)
Player
Activities
fl.py
Dec 15 08:51
kevin
Trash
kevin@ubuntu: ~
✅ Success Rate: 52.40% (262 / 500 episodes)
52.400000000000006
kevin@ubuntu:~$ python3 fl.py
✅ Success Rate: 47.80% (239 / 500 episodes)
47.8
kevin@ubuntu:~$ python3 fl.py
✅ Success Rate: 53.60% (268 / 500 episodes)
53.6
kevin@ubuntu:~$ python3 fl.py
✅ Success Rate: 53.00% (265 / 500 episodes)
53.0
kevin@ubuntu:~$ python3 fl.py
✅ Success Rate: 52.80% (264 / 500 episodes)
52.800000000000004
kevin@ubuntu:~$ python3 fl.py
^Z
[2]+ Stopped python3 fl.py
kevin@ubuntu:~$ python3 fl.py
✅ Success Rate: 50.00% (5 / 10 episodes)
50.0
[2]+ Killed python3 fl.py
kevin@ubuntu:~$ python3 fl.py
Frozen Lake

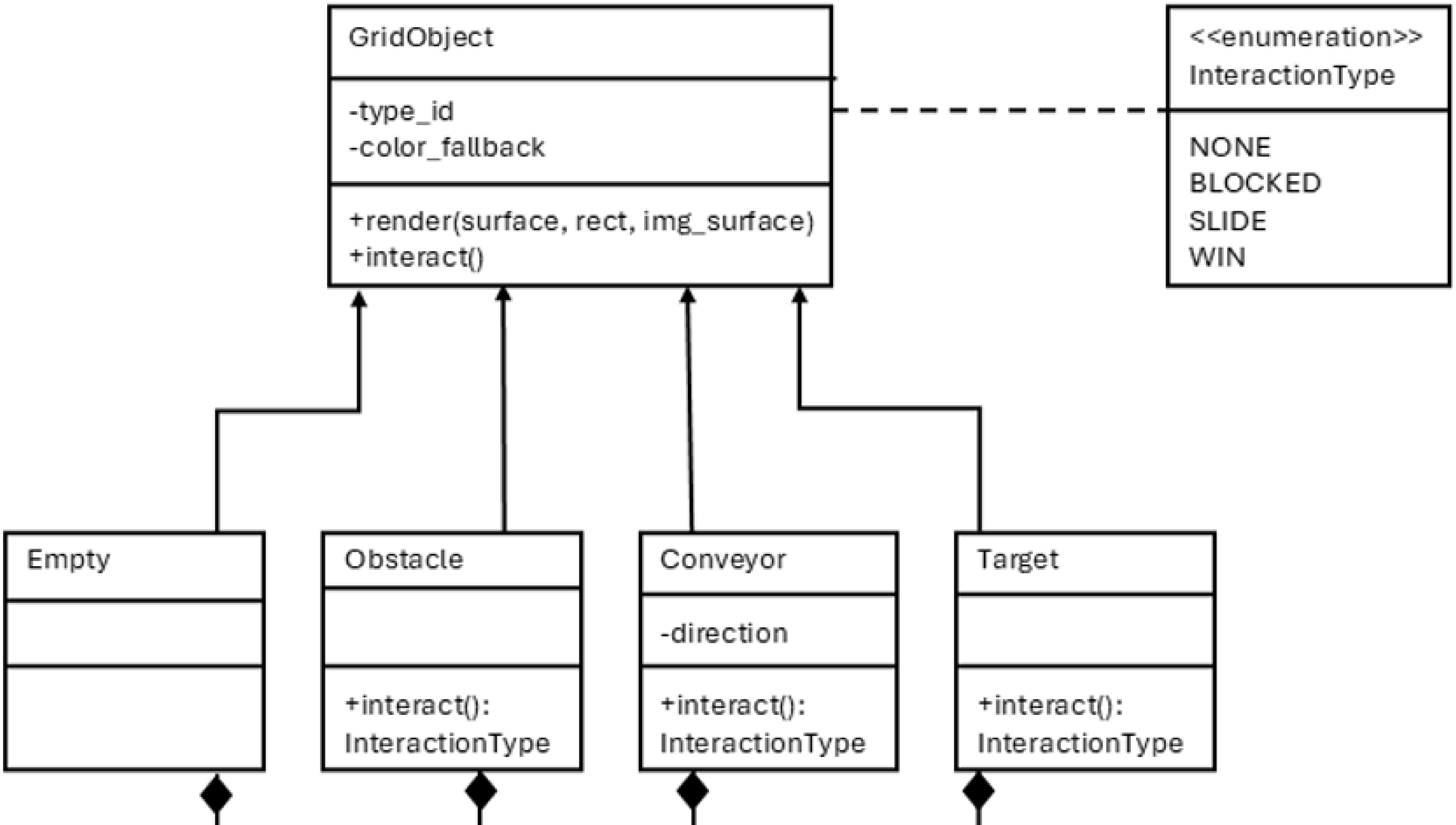
Demo

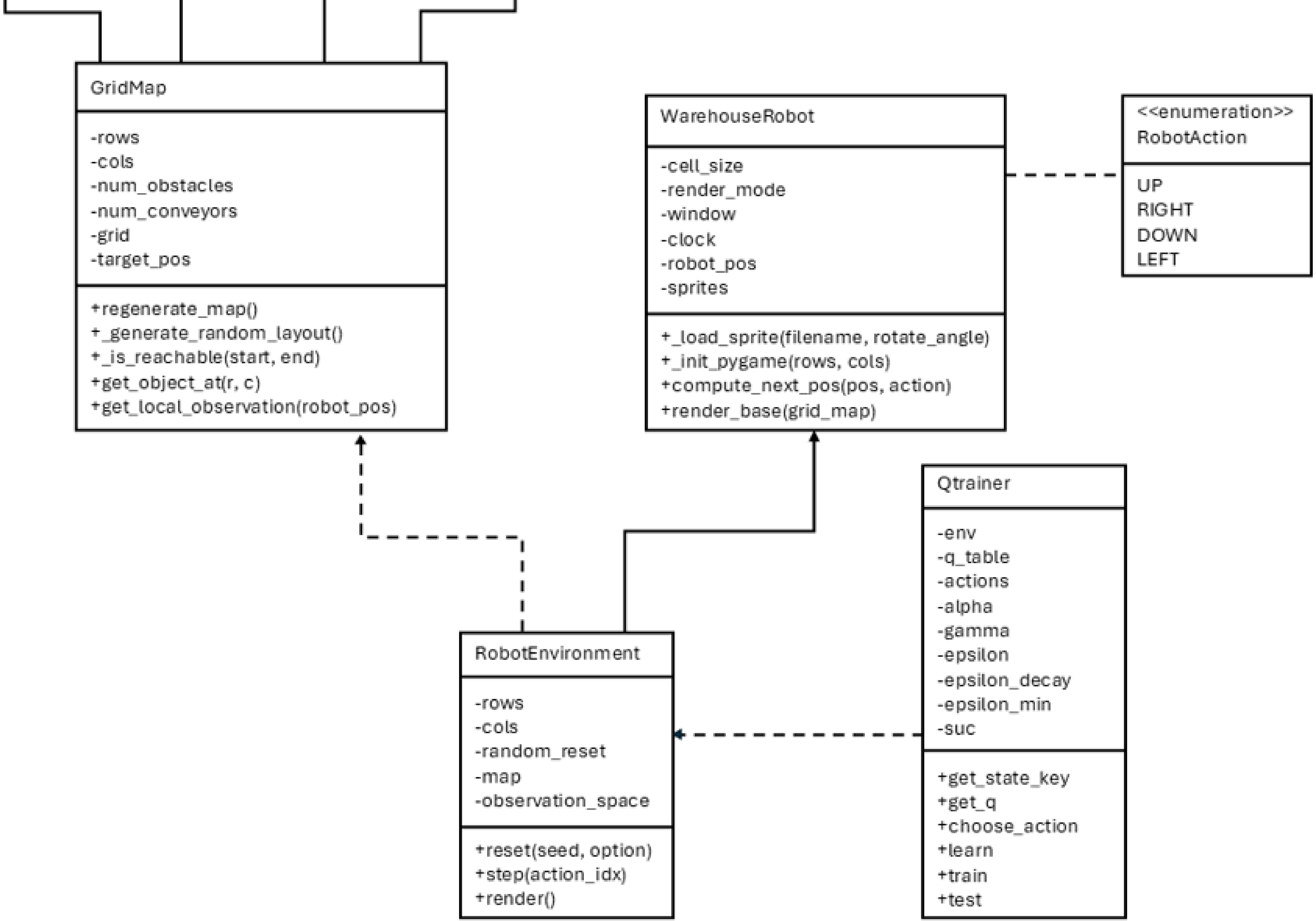


mnist.py
curve.py 1
vit.py 1
main.py
train_protonet.py
runall.py
protonet.py 7
C: > Users > Edwar > Downloads > Group-Project-main > Group-Project-main > part3 > main.py > ...
import gymnasium as gym
import robot_env # Trigger register
from trainer import QTrainer
if __name__ == "__main__":
# 1. 訓練
# random_reset=True: 每次地圖都不同，強迫學
env_train = gym.make('WarehouseRobot-v0',
trainer = QTrainer(env_train)
# 局部觀察的狀態組合約 1024 種，建議訓練 5000
trainer.train(episodes=5000)
env_train.close()
# 2. 測試
env_test = gym.make('WarehouseRobot-v0',
trainer.env = env_test
trainer.test(episodes=10)
env_test.close()
Warehouse Robot RL
PROBLEMS 9
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
python3.11
Episode 4000: Reward 20.0, Epsilon 0.14, Q-Size 121
Episode 4500: Reward -12.0, Epsilon 0.11, Q-Size 1222
Episode 5000: Reward 0.0, Epsilon 0.08, Q-Size 1227
--- Test Mode (With Stuck Detection) ---
Testing EP 1...
-> SUCCESS. Steps: 3, Stuck triggers: 0
Testing EP 2...
-> SUCCESS. Steps: 1, Stuck triggers: 0
Testing EP 3...

Project explaination

- 訓練人物走到終點(維持)
- 牆體(冰洞變體)
- 傳送帶(類似滑動)
- 局部觀察(強化學習效能)
- 隨機生成地圖(提升泛化能力)


GridObject
-type_id
-color_fallback
+render(surface, rect, img_surface)
+interact()
<<enumeration>>
InteractionType
NONE
BLOCKED
SLIDE
WIN
Empty
Obstacle
+interact():
InteractionType
Conveyor
-direction
+interact():
InteractionType
Target
+interact():
InteractionType


GridMap
-rows
-cols
-num_obstacles
-num_conveyors
-grid
-target_pos
+regenerate_map()
+_generate_random_layout()
+_is_reachable(start, end)
+get_object_at(r, c)
+get_local_observation(robot_pos)
WarehouseRobot
-cell_size
-render_mode
-window
-clock
-robot_pos
-sprites
+_load_sprite(filename, rotate_angle)
+_init_pygame(rows, cols)
+compute_next_pos(pos, action)
+render_base(grid_map)
<<enumeration>>
RobotAction
UP
RIGHT
DOWN
LEFT
Qtrainer
-env
-q_table
-actions
-alpha
-gamma
-epsilon
-epsilon_decay
-epsilon_min
-suc
+get_state_key
+get_q
+choose_action
+learn
+train
+test
RobotEnvironment
-rows
-cols
-random_reset
-map
-observation_space
+reset(seed, option)
+step(action_idx)
+render()